

Programación agéntica con Edge AI

Tres sesiones de 2 horas para dirigir a un agente de programación que construye, con skills propios y un plan visible, un asistente que corre modelos de IA en la propia laptop.

01

Herramientas y conceptos

Dictada el 08-10-2026. VS Code, Docker, OpenCode, agente, proveedor, modelo, parámetros, AGENTS.md, RTK, caveman, MCP, skills.

02

Asistente con skills y Edge AI

14 skills, plan de trabajo y obra en vivo. Chat local, voz a texto, flyers con texto real. Pruebas de uso y rendimiento.

03

WhatsApp y despliegue

Wireframe, UI y UX, afiliación por QR, webhook y bandeja sincronizada. Pruebas con teléfono real y despliegue con Docker.

PROYECTO GUÍA

Tecnosito: un asistente con avatar 3D que conversa con un modelo de lenguaje local, convierte voz a texto, genera imágenes y, en la sesión 3, atiende por WhatsApp. Cada equipo lo adapta con el corpus de su propio dominio.

EVALUACIÓN

Tarea 1 (entorno y primera skill), sesión 2 (plan, asistente, flyer, pruebas) y sesión 3 (wireframe, bot, pruebas, despliegue): 20 puntos cada una. Cierre: video de 3 minutos del asistente atendiendo por WhatsApp y una reflexión con tokens medidos.

Instalar las herramientas y entender qué hay detrás

Resumen de lo que ya quedó hecho. Diapositivas, ejemplos 01 a 07 y la tarea completa están en el LMS, clase `pa-01`.

OBJETIVOS

1. Instalar y comprobar VS Code, Docker y OpenCode en tu máquina.
2. Explicar qué es un agente de IA y qué son un proveedor, un modelo y sus parámetros.
3. Conocer reglas, ahorro de contexto, MCP y skills, y escribir tu primera skill.

ORDEN DE LA SESIÓN

#	BLOQUE	QUÉ SE VIO
1	VS Code	Instalación y comprobación con <code>code --version</code> .
2	Docker	Instalación y <code>docker run --rm hello-world</code> .
3	OpenCode y gateway	Instalación y conexión al gateway de la clase como proveedor compatible con OpenAI (<code>opencode.json</code> con la URL y la clave que entrega el instructor).
4	Conceptos	Agente (bucle que observa, decide y actúa con herramientas), proveedor, modelo, y los parámetros <code>temperature</code> y <code>max_tokens</code> en vivo.
5	Ahorrar contexto	Reglas del proyecto en <code>AGENTS.md</code> (<code>/init</code>); RTK comprime la salida de los comandos antes de que llegue al modelo (<code>rtk gain</code>); caveman como skill de respuestas cortas. Con la skill y el estilo del gateway a la vez el recorte se aplica dos veces: dejar uno.
6	MCP	Qué es y cómo se configura un servidor en OpenCode y en Claude Code.
7	Skills	Dónde van, front matter con <code>name</code> y <code>description</code> , cómo se cargan; práctica con la skill <code>landing-editorial</code> .

ENTREGABLE DE REFERENCIA



Landing page construida con el agente y una skill de landing: encabezado, propuesta de valor y botón de contacto por WhatsApp.

Tarea 1 · vence el 15-10 · 20 puntos

- Repositorio clase-01 con EVIDENCIA.md (salida de los tres comandos y captura de OpenCode respondiendo).
- Skill saludo en .opencode/skills/saludo/SKILL.md .
- REFLEXION.md : qué hizo el agente, qué aceptaste y qué no, y por qué.

Lo que la sesión 2 da por hecho

- VS Code, Docker y OpenCode funcionando contra el gateway.
- Saber qué es una skill y haber escrito una.
- RTK instalado y AGENTS.md en el proyecto.

Construir un asistente con skills, plan de trabajo y Edge AI

Resultado: cada equipo dirige a su agente para construir una variante funcional de Tecnosito con un modelo de lenguaje, reconocimiento de voz y generación de imágenes corriendo en local.

RESULTADOS DE APRENDIZAJE

1. Instalar y usar los 14 skills del taller con el modelo que cada uno tenga (OpenCode con el gateway, Claude Code, Codex, GPT, DeepSeek, GLM, Nemotron u Ollama).
2. Convertir un pedido vago en un plan de trabajo con tareas verificables y seguirlo en vivo con la pantalla de obra.
3. Elegir el modelo de imagen que cabe en su máquina y explicar por qué un modelo de difusión no escribe texto.
4. Probar el resultado con usuarios reales y medir tiempos.

DOMINIO DE CADA EQUIPO

Todos construyen el mismo asistente. Cada equipo lo adapta con el corpus de su propio dominio: el manual, reglamento o guía que usa en su trabajo, en 10 a 20 párrafos.

ANTES DE LA CLASE · 40 MINUTOS EN CASA, OBLIGATORIO

La red del aula no aguanta cinco descargas de 2,4 GB. Todo lo pesado se baja la noche anterior.

1. Enviar al grupo de WhatsApp: sistema operativo, RAM, GPU (modelo y VRAM) y CPU. El instructor responde con el nivel de imagen que te toca.
2. Instalar Ollama y bajar el modelo de chat: `ollama pull qwen3:1.7b`.
3. Descargar la carpeta del taller, crear el entorno y correr el diagnóstico:

```
cd taller-agentico
python3 -m venv .venv && source .venv/bin/activate # en Windows: .venv\Scripts\activate
pip install -r osito/backend/requirements.txt
python osito/backend/diagnostico.py
```

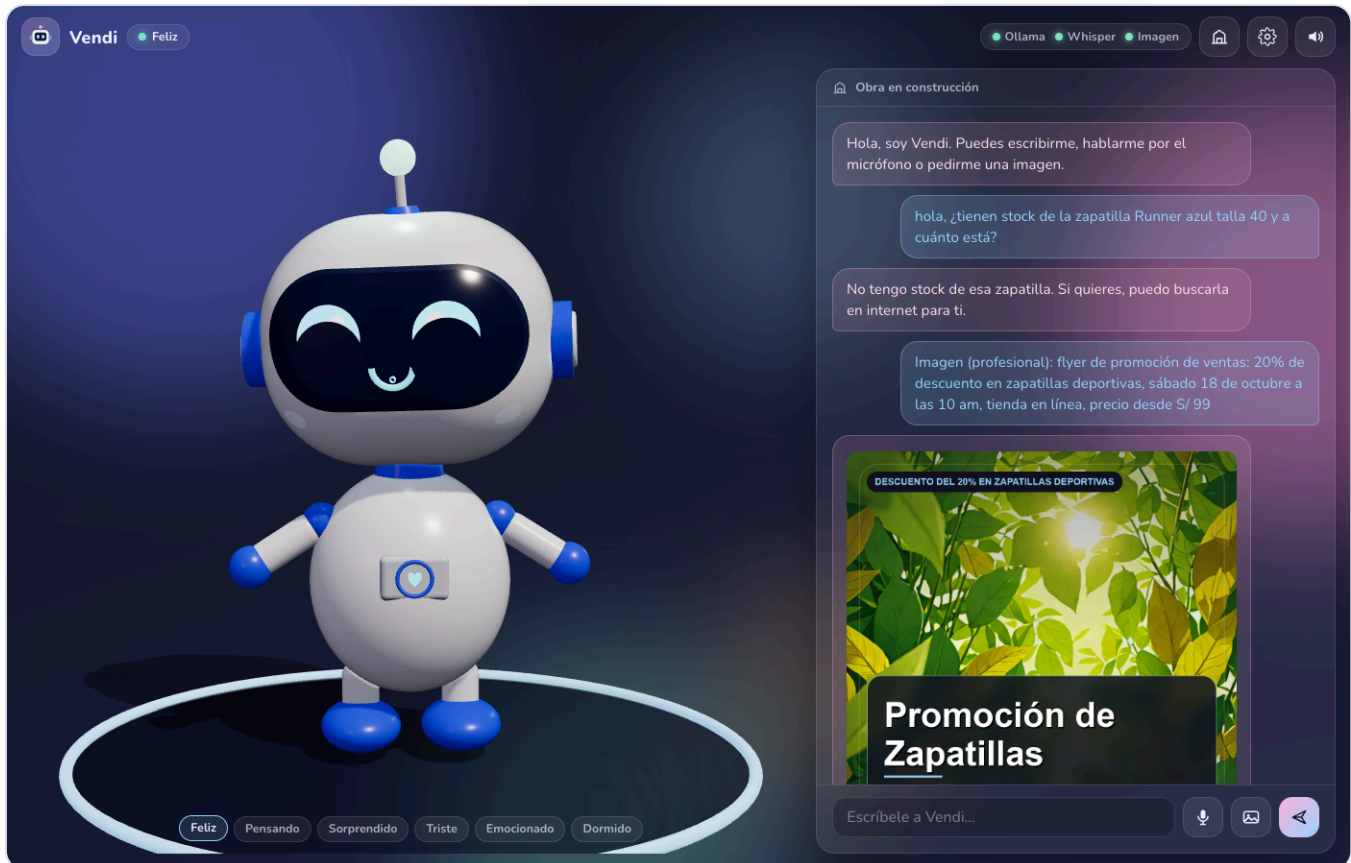
4. Descargar solo el modelo de imagen de tu nivel (comandos en `osito/README.md`). Con RTX serie 50, instalar antes torch con CUDA 12.8.
5. Traer el corpus en `manual.txt`, párrafos separados por línea en blanco.

Quien llegue sin modelo de imagen trabaja igual: el sistema cae a modo demo.

CRONOGRAMA

TIEMPO	BLOQUE	QUÉ PASA	ENTREGABLE
0:00 a 0:10	Apertura	Repaso de 2 minutos de la sesión 1. Qué es Edge AI. Demo de Tecnosito: pregunta por voz, respuesta hablada, flyer generado.	Ninguno
0:10 a 0:25	Skills	<code>bash</code> <code>instalar.sh</code> copia los 14 skills; <code>AGENTS.md</code> y <code>prompts/</code> para quien usa OpenCode u Ollama. Activar modo piedra. Pedir un plan de proyecto sin skill y luego con <code>planificacion-proyecto</code> ; comparar.	Skills instalados y plan de media página
0:25 a 0:40	Plan y obra	Con <code>plan-de-trabajo</code> , escribir <code>PLAN_TRABAJO.md</code> : 6 tareas de 15 minutos con criterio verificable. <code>obra.py</code> <code>servir</code> y la pantalla de construcción visible en cada equipo.	Plan con 6 tareas, obra al 0 %
0:40 a 1:10	Construcción	El agente ejecuta el plan con los skills. Cargar el corpus en <code>backend/faq.json</code> , cambiar nombre, saludo y ánimo. Cada tarea cerrada se marca con <code>obra.py</code> <code>paso</code> .	Asistente respondiendo preguntas del dominio con voz
1:10 a 1:30	Edge AI	Micrófono a texto con whisper. Flyer del evento del equipo: revisar los datos que extrajo el LLM, cambiar uno a propósito, regenerar. Por qué el modelo de difusión no escribe y cómo se resolvió. Comparar niveles rápida, buena y máxima.	Flyer con datos correctos
1:30 a 1:50	Pruebas	Pruebas de uso cruzadas con <code>pruebas-de-uso</code> : 5 minutos con el asistente de otro equipo y un guion de 3 tareas. Medir con <code>pruebas-de-rendimiento</code> : chat, voz a texto e imagen.	Hoja de hallazgos y tabla de tiempos
1:50 a 2:00	Cierre	Un minuto por equipo. Lecciones. Tarea 2 y preparación de la sesión 3.	Obra al 100 %

ENTREGABLE DE REFERENCIA



Asistente de ventas con avatar 3D: responde con el modelo local, habla con subtítulos y genera un flyer de promoción a pedido, todo en la laptop.

NOTAS POR BLOQUE

Skills. `skills/_perfiles/MODELOS.md` dice dónde pegar cada skill según el modelo. Con OpenCode y el gateway se usa `AGENTS.md`; con Ollama o DeepSeek, `prompts/<skill>.txt` como system prompt. El skill `comandos-optimizados` y RTK se complementan: uno pide comandos baratos, el otro comprime lo que igual sale.

Plan y obra. Regla: ninguna tarea sin forma de probarla. Seis tareas sugeridas: cargar el corpus; nombre, saludo y ánimo; tres preguntas del dominio; micrófono; flyer; pruebas de uso. La pantalla de obra se refresca cada 2 segundos y deja ver avanzar al agente sin leer su terminal.

Construcción. El respaldo por embeddings devuelve "No estoy seguro de eso todavía" bajo el umbral de similitud; es correcto, no un error. Vigilar que el agente marque cada paso en obra y que no invente respuestas fuera del corpus.

Edge AI. La tubería del flyer tiene tres piezas: el LLM extrae los datos, el modelo de difusión pinta solo el fondo, el código compone la tipografía. Un regex protege fecha, hora y precio del LLM. Tiempos de referencia en Mac Apple Silicon de 24 GB: rápida 9 s, buena 7 s en caliente, máxima 45 s. Sin GPU usable, rápida tarda de 25 a 60 s y es normal.

Pruebas. Un dato alterado por el LLM que el usuario no detecta es severidad 4. El backend imprime la duración de cada petición en su log.

Evaluación · 20 puntos

PLAN_TRABAJO.md con 6 tareas verificables	4
Asistente respondiendo 3 preguntas del corpus, con voz y subtítulo	6
Flyer con título, fecha, hora y precio exactos	4
Hoja de hallazgos: 3 tareas, severidades, una mejora	4
Obra al 100 % con horas reales	2

Tarea 2 y preparación de la sesión 3

1. Ampliar el corpus del asistente con 10 preguntas y respuestas más de tu dominio en `faq.json`, probar 3 de ellas por voz y entregar una captura de cada respuesta con 5 líneas de lo que aprendiste.
2. Para la sesión 3: `docker pull` de la imagen de Evolution API y conseguir un número de WhatsApp de pruebas del equipo que no sea el personal de nadie.

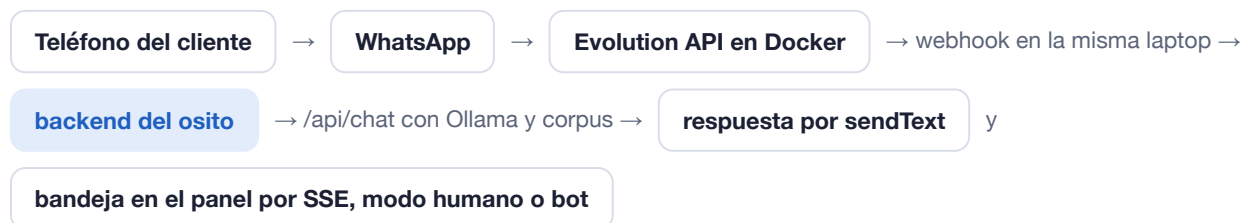
Llevarlo a WhatsApp: wireframe, UI, desarrollo, pruebas y despliegue

Resultado: el asistente de cada equipo atiende mensajes de WhatsApp con su número de pruebas, las conversaciones se ven sincronizadas en el panel de Tecnosito y el conjunto queda desplegado con Docker y comprobado.

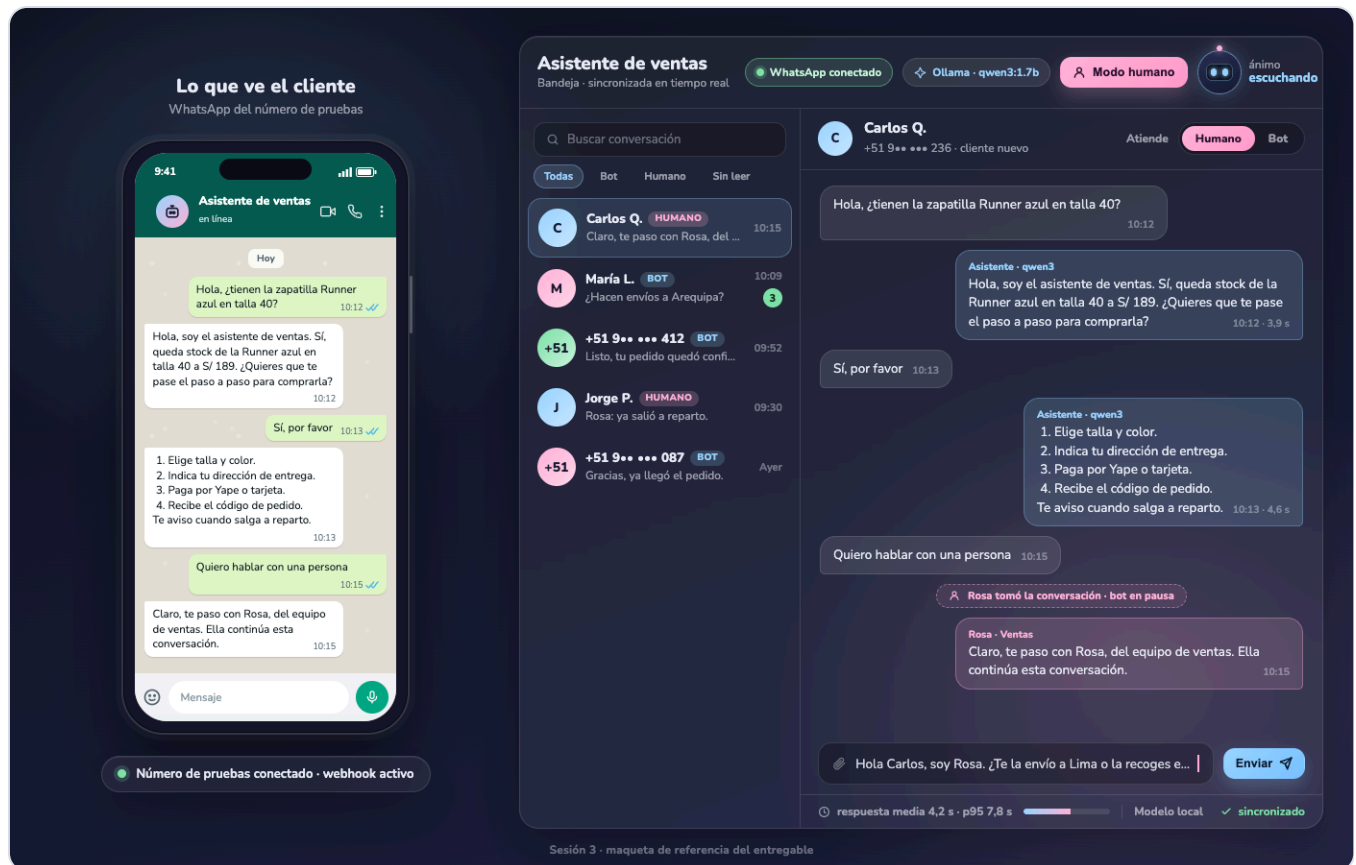
RESULTADOS DE APRENDIZAJE

1. Diseñar antes de programar: wireframe de baja fidelidad, flujo feliz y flujos de error, y pasarlo a UI con los skills de diseño, responsividad y accesibilidad.
2. Afiliar un número de WhatsApp a una API local (Evolution API) por código QR y recibir sus mensajes por webhook.
3. Sincronizar el bot con el asistente: cada mensaje entra por el webhook, lo responde el modelo local y se refleja en el panel; un humano puede tomar la conversación.
4. Entender qué es una herramienta del agente y ver un MCP en acción; usar skills, RTK y el modo caveman o piedra para que el agente trabaje barato y rápido.
5. Probar con un teléfono real y con pruebas automáticas, y desplegar con Docker Compose con una comprobación de salud.

ARQUITECTURA



ENTREGABLE DE REFERENCIA



Maqueta del entregable: el asistente de ventas atiende por WhatsApp y la bandeja web muestra las conversaciones con interruptor humano o bot.

ANTES DE LA CLASE · 30 MINUTOS EN CASA, OBLIGATORIO

1. Docker corriendo y la imagen de Evolution API ya descargada.
2. Docker con la red por defecto: Evolution API corre en un contenedor y llama al backend de la laptop por `host.docker.internal`. No hace falta nada fuera de la laptop.
3. Número de WhatsApp de pruebas del equipo, con el teléfono presente en clase, y un segundo teléfono para escribirle.
4. RTK instalado y los 14 skills en su sitio. Releer en el LMS qué es una herramienta y qué es un MCP (sesión 1, bloque 6); en clase el instructor muestra uno funcionando, no hay que instalar nada.
5. Tarea 2 entregada: el asistente ya responde el corpus del equipo.

Reglas de uso de WhatsApp en clase

- Solo números de pruebas del equipo. Nadie afilia su número personal.
- Quien escriba al bot debe saber que habla con un bot: el primer mensaje lo dice.
- Nada de envíos masivos, listas de contactos ni mensajes a terceros fuera del aula. Un número bloqueado no se recupera en clase.

CRONOGRAMA

TIEMPO	BLOQUE	QUÉ PASA	ENTREGABLE
0:00 a 0:10	Apertura	Qué se construye hoy y la arquitectura. Demo: un mensaje desde un teléfono, el osito cambia de ánimo, responde, y la conversación aparece en la bandeja.	Ninguno
0:10 a 0:25	Wireframe	Con <code>ux-flujos</code> y <code>ui-componentes</code> , wireframe en gris de tres pantallas: bandeja de conversaciones, conversación con interruptor humano o bot, estado del número (QR, conectado, caído). Flujo feliz y dos de error: número desconectado; el modelo no responde.	<code>WIREFRAME.html</code> o boceto en Excalidraw, y <code>FLUJOS.md</code>
0:25 a 0:40	UI y UX	Pasar el wireframe a UI con <code>diseño-visual</code> , <code>responsividad</code> y <code>accesibilidad</code> , en el estilo del osito. Revisar a 393 px. Microcopy en español claro.	Panel "Bandeja" visible, aún sin datos
0:40 a 1:05	Afiliación	Levantar Evolution API con <code>docker compose up -d</code> , crear la instancia, escanear el QR con el teléfono de pruebas. Registrar el webhook apuntando a <code>http://host.docker.internal:8766/api/whatsapp/webhook</code> . Prueba: "hola" desde el segundo teléfono y ver el JSON en el log, con RTK encendido.	Número conectado y primer webhook recibido
1:05 a 1:30	Desarrollo	Con <code>plan-de-trabajo</code> y obra, pedir al agente: endpoint <code>/api/whatsapp/webhook</code> que llama a <code>/api/chat</code> y responde por <code>sendText</code> ; eventos SSE en <code>/api/whatsapp/eventos</code> hacia la bandeja; interruptor "modo humano" por conversación; el osito pasa a <i>escuchando</i> al recibir y a <i>hablando</i> al responder. El agente trabaja en modo piedra.	Bot respondiendo por WhatsApp y bandeja sincronizada
1:30 a 1:45	Pruebas	Teléfono real: tres mensajes del guion (pregunta del corpus, pregunta fuera del corpus, pedir hablar con una persona). Automáticas: <code>curl</code> con un JSON de webhook grabado. Demostración del instructor: el agente prueba el panel con un MCP de navegador, para ver qué es una herramienta en acción. Latencia de mensaje a respuesta con <code>pruebas-de-rendimiento</code> .	Hoja de hallazgos y latencia p50 y p95
1:45 a 2:00	Despliegue	<code>docker compose</code> con backend, Evolution API y variables en <code>.env</code> ; imagen horneada; <code>/api/salud</code> en verde; un mensaje real respondido; captura final. Opcional: el mismo compose en el servidor del curso si el instructor asigna ruta. Cierre y tarea 3.	Stack desplegado con salud verde y obra al 100 %

NOTAS POR BLOQUE

Wireframe. Gris, sin color ni tipografía final: decide qué hay en cada pantalla y en qué orden. Los flujos de error se escriben antes que el feliz: número desconectado y modelo caído son lo que pasará en clase.

UI y UX. El interruptor humano o bot va en la conversación, no escondido en ajustes. Todo lo que el bot envía se ve en la bandeja; nada sale sin registro.

Afiliación. Como Evolution corre

Desarrollo. Filtrar los mensajes propios y los de grupos. Guardar las conversaciones en memoria o en un JSON. Responder siempre, aunque sea "No estoy seguro de eso todavía, te paso con una persona". Una herramienta es una función que el agente puede llamar; un MCP es la forma estándar de ofrecerle varias. En clase se ve una sola vez, en manos del instructor.

Pruebas. Latencia esperada con qwen3:1.7b en CPU: de 3 a 8 segundos. Más de 15 segundos sin respuesta es hallazgo de severidad 3.

Despliegue. Desplegar es sincronizar, hornear, levantar y comprobar; nunca solo hacer commit. `docker compose config` antes de `up`. Secretos solo en `.env`, fuera de git.

Evaluación · 20 puntos		
Wireframe de 3 pantallas y FLUJOS.md		3
Panel "Bandeja" responsivo y accesible		3
Número afiliado y webhook recibido		3
Bot respondiendo con bandeja sincronizada y modo humano		5
Pruebas: hallazgos, curl del webhook, latencia medida		3
Despliegue con Docker Compose y salud verde		3

Tarea 3 · cierre del curso

Video de 1 minuto con el asistente del equipo respondiendo 3 preguntas por WhatsApp a un compañero de otro equipo. Más REFLEXION.md de 10 líneas: qué hizo el agente solo, qué hubo que corregirle y qué aprendiste del curso.

Si algo falla

- Ollama no responde: el asistente cae al respaldo por embeddings; el chip de salud lo muestra en gris. Se puede seguir.
- El micrófono no funciona: Chrome solo lo permite en `localhost`, no por IP.
- La imagen tarda más de un minuto: bajar a calidad rápida o dejar el modo demo.
- Puerto ocupado: `OSIT0_PUERTO=8766 bash osito/arrancar.sh` y `python obra/obra.py servir --puerto 8797`.
- Descarga de un modelo quieta: cancelar y repetir con `HF_HUB_DISABLE_XET=1`.
- El webhook no llega: comprobar que el backend está en 8766 y que la URL registrada usa `host.docker.internal`, no `localhost` (dentro del contenedor, `localhost` es el propio contenedor).
- El QR no aparece: la instancia no existe o el contenedor no sale a internet.